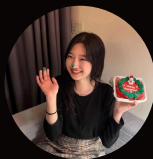
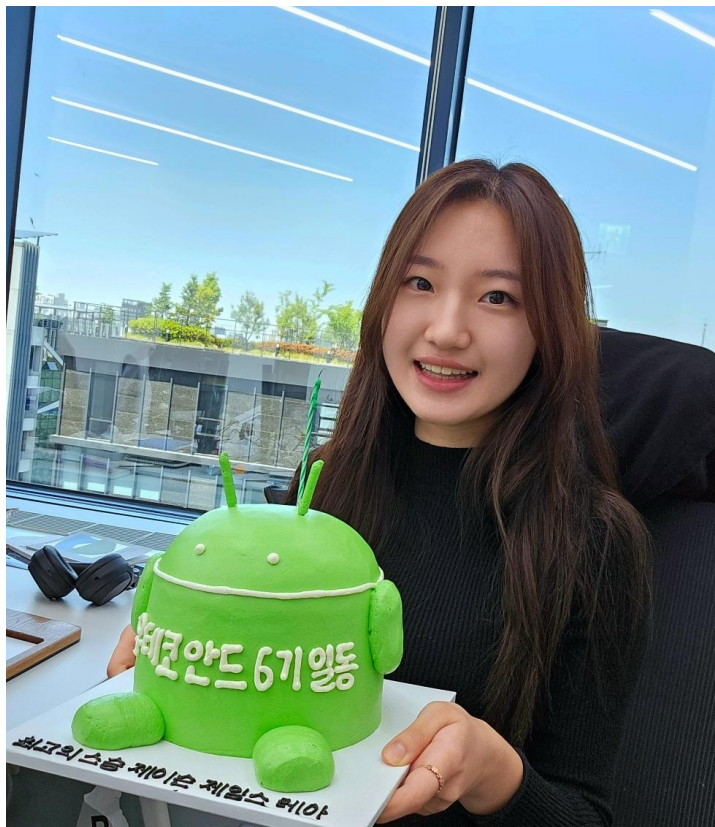


Compose UI 컴포넌트 설계와 테스트

김수현 / 우아한형제들





GitHub @wisemuji

- ❑ 우아한형제들 Android Dev/Educator
- ❑ NEXTSTEP Educator/Code Reviewer
- ❑ DroidKnights Conference Organizer
- ❑ GDG Korea Android Organizer

이야기할 내용

- ❑ Compose UI 컴포넌트 설계의 중요성
- ❑ React로 살펴보는 선언형 UI 컴포넌트 설계 노하우
- ❑ 사례로 알아보는 UI 컴포넌트 설계 팁
- ❑ 사례로 알아보는 UI 컴포넌트 테스트

다루지 않는 내용

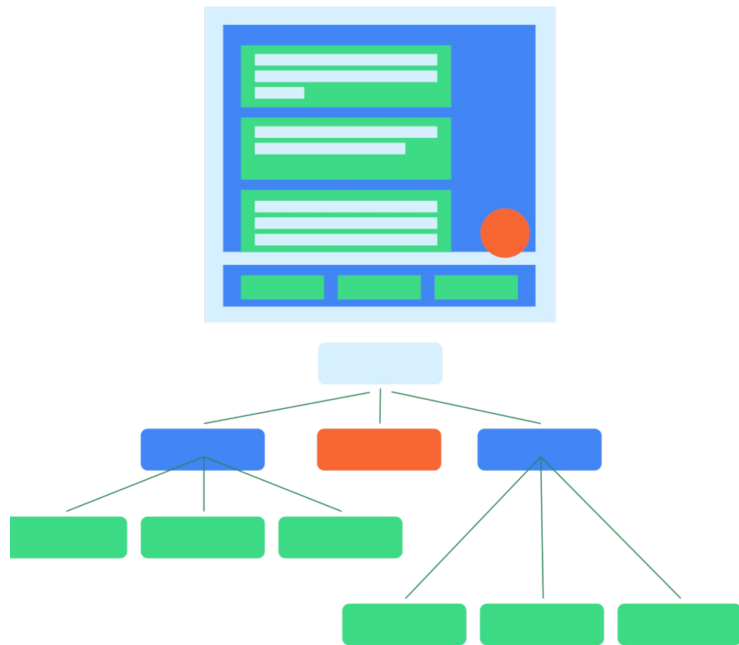
- ❑ UI 테스트 프레임워크 사용법
- ❑ 웹 프론트엔드 및 React 개념
- ❑ 컴포넌트 아키텍처 패턴
- ❑ Compose Stability

“구현체가 아닌 인터페이스에 의존”

Compose UI 컴포넌트 설계의 중요성

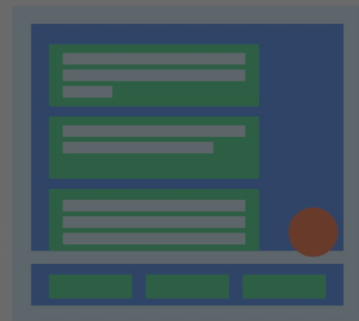
컴포넌트를 나누는 기준

- ❑ 재사용 가능성
- ❑ 테스트 가능성
- ❑ 하나의 역할/책임
- ❑ 변경에 유연함
- ❑ ...



컴포넌트를 나누는 기준

- ❑ 재사용 가능성
- ❑ 테스트 가능성
- ❑ 하나의 역할/책임
- ❑ 변경에 유연함
- ❑ ...



Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive - Android 실전편

Track 01

17:00 발표



김준비

드로이드나이트 앱

github.com/droidknights/DroidKnightsApp

❑ Case1: 화면 단위 구현

❑ Case2: 컴포넌트 단위 구현

Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive - Android 실전편

Track 01

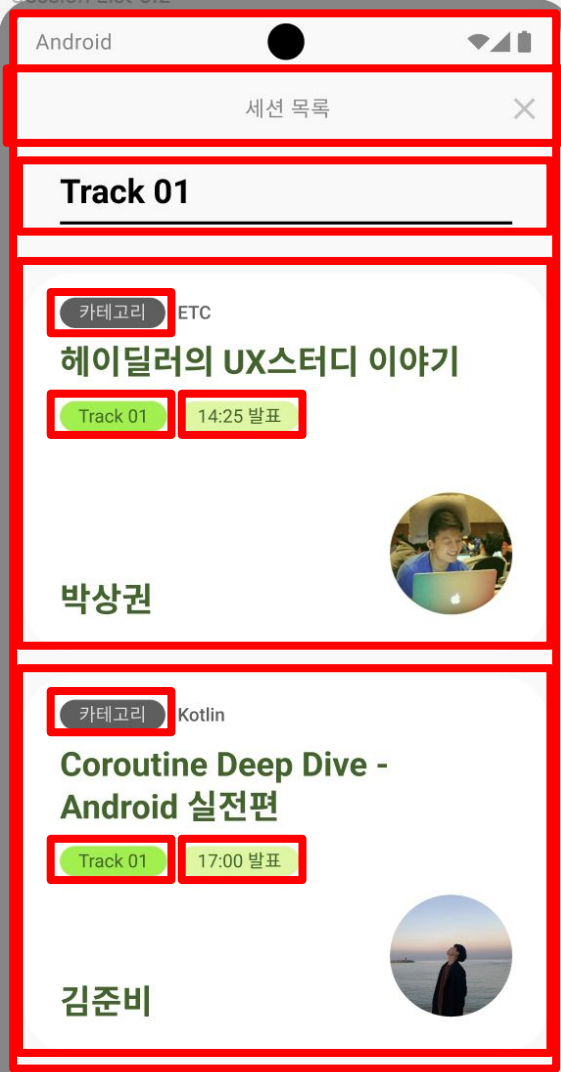
17:00 발표



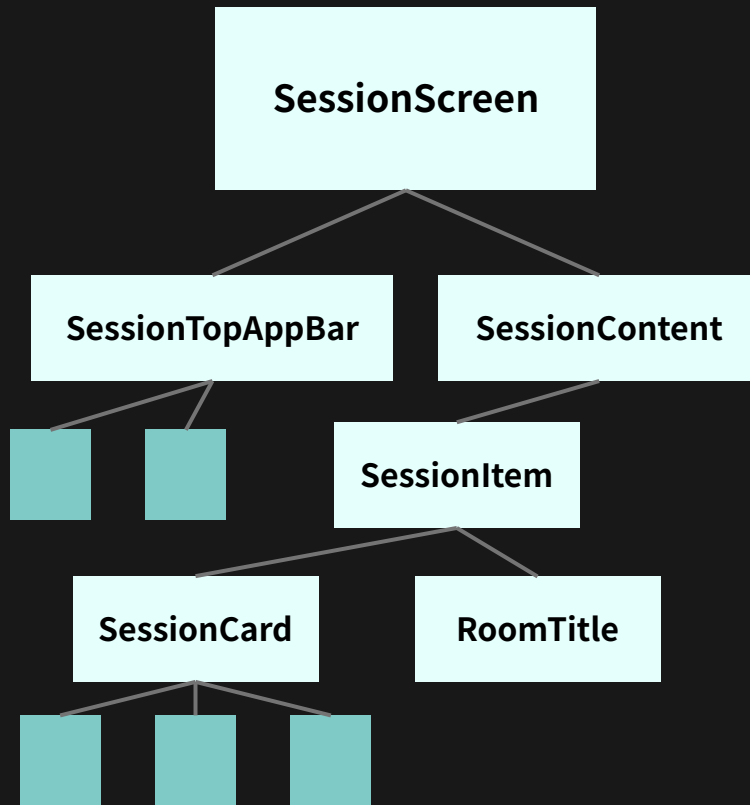
김준비

Case1: 화면 단위 구현

SessionScreen



Case2: 컴포넌트 단위 구현



Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01 14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive -
Android 실전편

Track 01 17:00 발표



김준비

변경사항이 생긴다면?

- ❑ 새로운 기능이 추가된다면?
- ❑ 디자인 스타일이 바뀐다면?
- ❑ 아이템의 배치가 변경된다면?

Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive -
Android 실전편

Track 01

17:00 발표

북마크



김준비

변경사항이 생긴다면?

- ❑ 새로운 기능이 추가된다면?
- ❑ 디자인 스타일이 바뀐다면?
- ❑ 아이템의 배치가 변경된다면?

북마크 기능 추가



Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive - Android 실전편

Track 01

17:00 발표

북마크



김준비

Case1: 화면 단위 구현

SessionScreen

Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive - Android 실전편

Track 01

17:00 발표

북마크



김준비

어디부터 건드려야 하지? 🤯
변경사항이 전체 화면에 전파

Case1: 화면 단위 구현

SessionScreen

Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive - Android 실전편

Track 01

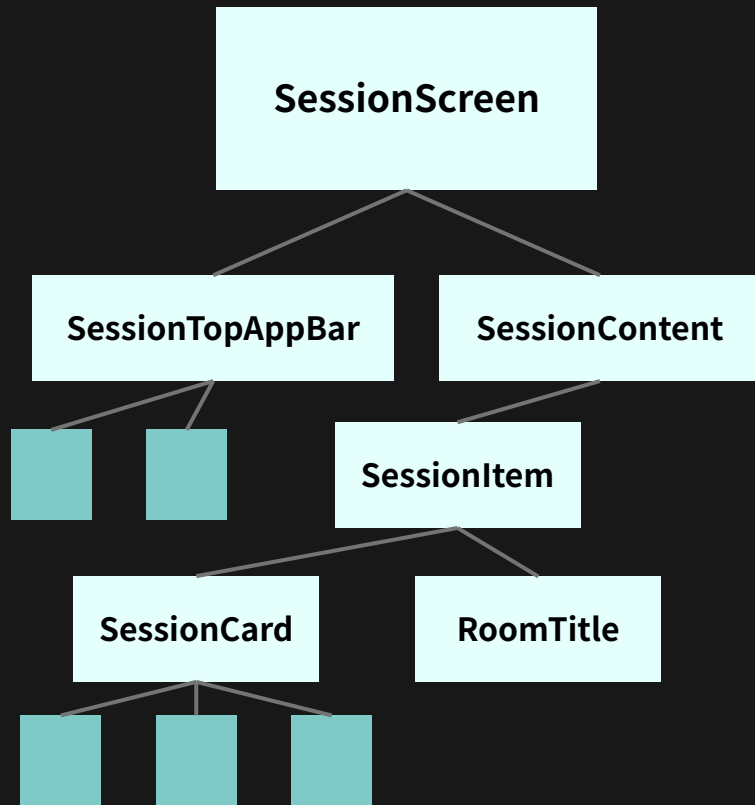
17:00 발표

북마크



김준비

Case2: 컴포넌트 단위 구현



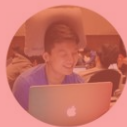
Track 01

카테고리 ETC

헤이딜러의 UX스터디 이야기

Track 01

14:25 발표



박상권

카테고리 Kotlin

Coroutine Deep Dive - Android 실전편

Track 01

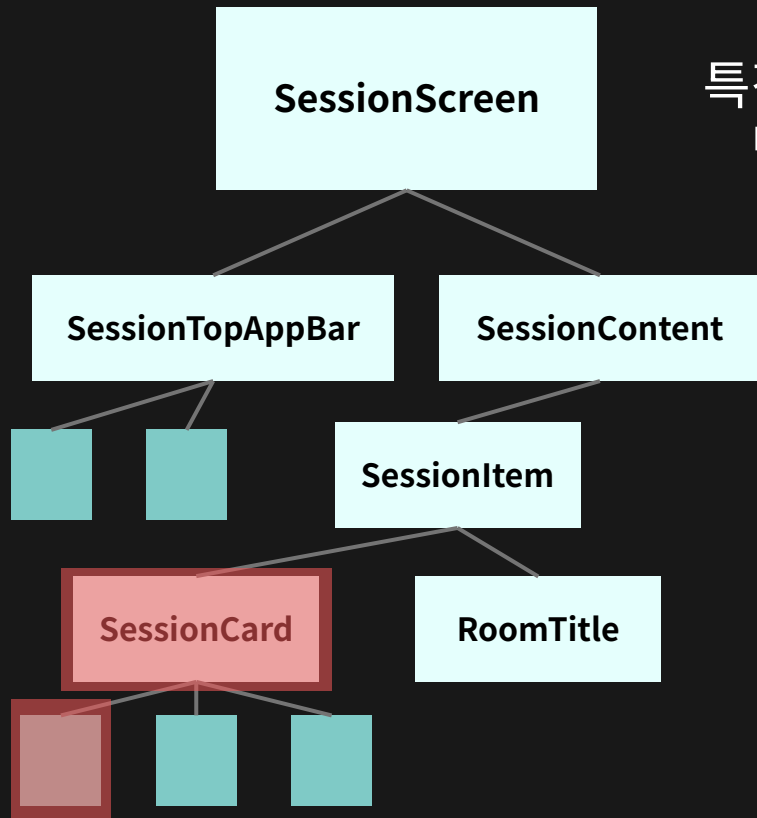
17:00 발표

북마크



김준비

Case2: 컴포넌트 단위 구현



특정 컴포넌트에만
변경사항 전파

API Guidelines for @Composable components in Jetpack Compose

(중략)...

Before you create a component

When creating a new component:

- Make sure there's a single problem the component solves. Split components into subcomponents and building blocks until each solves a single problem users have.
- Make sure you need a component and it brings value that justifies the long term support and evolution of its APIs. Often developers might find that it is easier for users to write the component code themselves so they can adjust later.

Component's purpose

Consider the value a new component adds and the problem it solves. Each component should solve only **one** problem, and each problem should be solved in **one** place. If your component solves more than one problem look for opportunities to split it into layers or subcomponents. With the benefit of smaller, concise and use case targeted API comes the easy of use and clear understanding of the component contract.

하나의 컴포넌트는 하나의 문제만 해결해야 한다.

컴포넌트가 하나 이상의 문제를 해결한다면

하위 계층 또는 하위 컴포넌트로 분할하는 것을 고려하라.

**작고 간결한 API로 디자인할수록 사용하기 쉽고,
컴포넌트 인터페이스를 명확하게 이해할 수 있다.**

UI 컴포넌트 설계의 중요성

- ❑ Android View에서는 화면(Activity, Fragment) 단위의 구현이 일반적이었으나, Compose에서는 **선언적으로 작성된 컴포넌트로 쪼개어** 설계할 수 있다.
- ❑ 적절한 컴포넌트 단위를 나누어 **변경사항에 유연한 설계**를 지향하자.
- ❑ 결합도를 낮추고 응집도를 높이자.

React로 살펴보는 선언형 UI 컴포넌트 설계 노하우

선언형 UI 패러다임 선배들

- ❑ React - **2013** JSConf US 오픈소스화
- ❑ Flutter - **2015** 닥트 개발자 서밋 첫 공개
- ❑ SwiftUI - **2019** WWDC 첫 소개
- ❑ Jetpack Compose - **2021** Stable 버전 정식 출시

선언형 UI 패러다임 선배들

- ❑ React - **2013** JSConf US 오픈소스화
- ❑ Flutter - **2015** 닥트 개발자 서밋 첫 공개
- ❑ SwiftUI - **2019** WWDC 첫 소개
- ❑ Jetpack Compose - **2021** Stable 버전 정식 출시



React - 컴포넌트 설계 노하우 多

Google

react component 설계



전체

이미지

동영상

뉴스

쇼핑

더보기

도구



velog

<https://velog.io> > designing-complex-components-flexibly

복잡한 컴포넌트 유연하게 설계하기

2023. 4. 7. — React를 사용할 때 css-in-js 는 React 컴포넌트와 같은 레벨로 추상화가 되기에 컴포넌트와 함께 유연하게 사용할 수 있고 여러 컴포넌트에서 재사용하기 ...



stevy.dev

<https://www.stevy.dev> > react-design-guide

리액트 설계 가이드 | Stevy's wavyLog

리액트 컴포넌트를 딱 한가지 역할만 수행시키기 위해 설계 하기 위해서는 순수함수 의 특성과 같이 동일한 props를 받으면 같은 JSX 리턴하는 순수함수 로 이뤄져야 한다.



velog

<https://velog.io> > 더-나은-컴포넌트-설계하기

더 나은 컴포넌트 설계하기

2022. 6. 26. — 의도에 집중한 프로그래밍 방법이다. 설계만 잘 하면 React는 데이터가 변경됨에 따라 적절한 컴포넌트만 효율적으로 갱신하고 렌더링한다. · 상속인 아닌 ...



티스토리

<https://hong-jh.tistory.com> > entry, 리액트-컴포넌트-...

리액트 컴포넌트 설계 방법 - 홍준혁 - 티스토리

2022. 6. 22. — 리액트 설계 방법에 대한 이해가 부족하다고 생각되어 공부하면서 글을 정리합니다. 개발을 하다 보면 앱이 커지고 그릴수록 컴포넌트의 수가 많아 ...



티스토리

<https://sugoring-it.tistory.com> > ...

리액트 React | 프로젝트 구조 설계 패턴 - 슈고릿 - 티스토리

Google

react component design best practices



전체

이미지

쇼핑

동영상

뉴스

더보기

React best practices

- Keep components small and function-specific. ...
- Put CSS in JavaScript. ...
- Use a linter. ...
- Comment only where necessary. ...
- Use capitals for component names. ...
- Separate stateful aspects from rendering. ...
- The code should be testable. ...
- All files related to any component should be in a single folder.

더보기 · 2023. 12. 20.



Fireart Studio

<https://fireart.studio> > ... > React

10 React Best Practices to Improve Your Code | Fireart Studio

추천 스니펫 정보



DEV Community

<https://dev.to> > blossom, 8-best-practices-for-reactjs-co...

8 Best Practices for React.js Component Design

2023. 7. 17. — 8 Best Practices for React.js Component Design · Use libraries like Jest and cypress to write unit tests for your components. · Test props and ...



freeCodeCamp

<https://www.freecodecamp.org> > news > best-practices-f...

2024

DROID
KNIGHTS

Jetpack Compose는?

Google

compose component design best practices



전체 이미지 쇼핑 동영상 뉴스 더보기



Android Developers

<https://developer.android.com> > ... > Core areas > UI

Follow best practices | Jetpack Compose

2024. 4. 11. — Follow **best practices** · Use remember to minimize expensive calculating layout keys · Use derivedStateOf to limit recompositions · Defer ...



Medium

<https://medium.com> > creating-cohesive-ui-with-jetpack...

Creating Cohesive UI with Jetpack Compose: 10 Best ...

2023. 4. 4. — By following these **best practices**, you'll create a more cohesive and ma using Jetpack Compose, ultimately leading to a better user ...



Android Developers

<https://developer.android.com> > ... > Core areas > UI

Custom design systems in Compose

Best practices. Style guidelines. Compose API guidelines ... Using Material compon Implementing a fully-custom **design system**. Using Material components.



Medium

<https://medium.com> > make-android > mastering-jetpac...

Best Practices and Architectural Patterns | Make Android

2023. 9. 20. — In this article, we will explore the **best architecture** for Jetpack Comp with examples and explanations. Why Architectural Patterns ...

관련 질문

Google

compose 컴포넌트 설계



전체 이미지 쇼핑 동영상 뉴스 더보기

도구



velog

<https://velog.io> > Android-Jetpack-Compose-UI-Compon...

[Android] Jetpack Compose UI Component

2024. 1. 10. — Jetpack **Compose UI**란? 선언적 UI 패러다임을 기반으로 하여, 개발자가 보다 직관적이고 간결한 방식으로 안드로이드 앱의 UI를 설계할 수 있게 해주 ...



티스토리

<https://comper.tistory.com> > entry > Android-좋은-Vie...

[Android] 좋은 View, 좋은 Component 를 설계하는 방법과 ...

2022. 10. 6. — 바꿔 말하면, 아무리 **Compose**를 사용하더라도 **Compose 컴포넌트**에 상태를 마구 저장하면 **Compose**라는 컴데기를 쓴 명령형 UI 프로그래밍을 하는 것이 되는 ...



티스토리

<https://devel-yong.tistory.com> > entry > Composable-A...

Composable Component API Guideline :: 아녀 똥인데요??

2024. 2. 29. — 하위 수준의 'building block'과 '서브 컴포넌트'는 간단하고 특정한 단일 기능을 가지며, 다른 컴포넌트와 쉽게 결합될 수 있도록 설계됩니다. 예를 들어 ...



velog

<https://velog.io> > my-compose-architecture

사이드 프로젝트에서 Jetpack Compose 를 이용한 ...

2022. 4. 22. — Jetpack **Compose** 적용기. 구글 문서에서 확인할 수 있듯이 **Compose**에서는 상태 호이스팅을 강조하고 있다. 상태는 위에서 아래로 흐르고, 이벤트는 아래 ...



Medium

<https://medium.com> > compose-도입을-고려하시나요...

Compose 도입을 고려하시나요?

2024

**DROID
KNIGHTS**

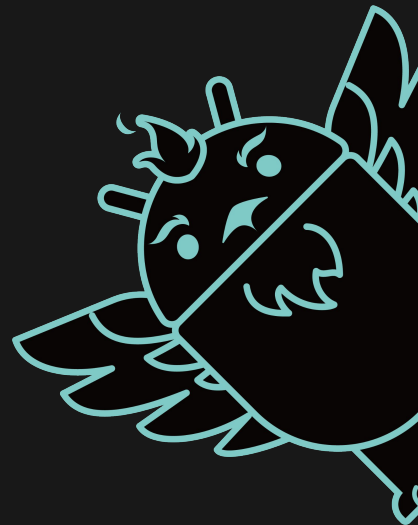
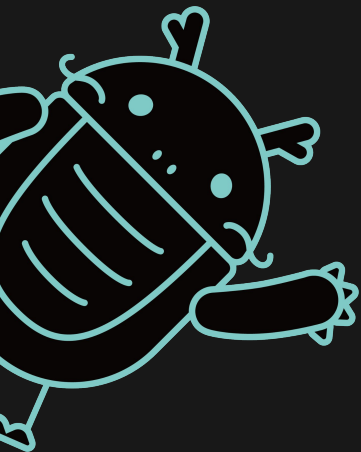
React 컴포넌트 설계 Best Practices

React 개발자들이 공유하는 핵심 설계 원칙은 Compose UI에도 적용 가능

- ❑ 함수형 컴포넌트
- ❑ 작고 재사용 가능한 컴포넌트
- ❑ UI와 비즈니스 로직의 분리
- ❑ Atomic Design

선언형 UI 패러다임은 서로 밀접하게 닿아있다

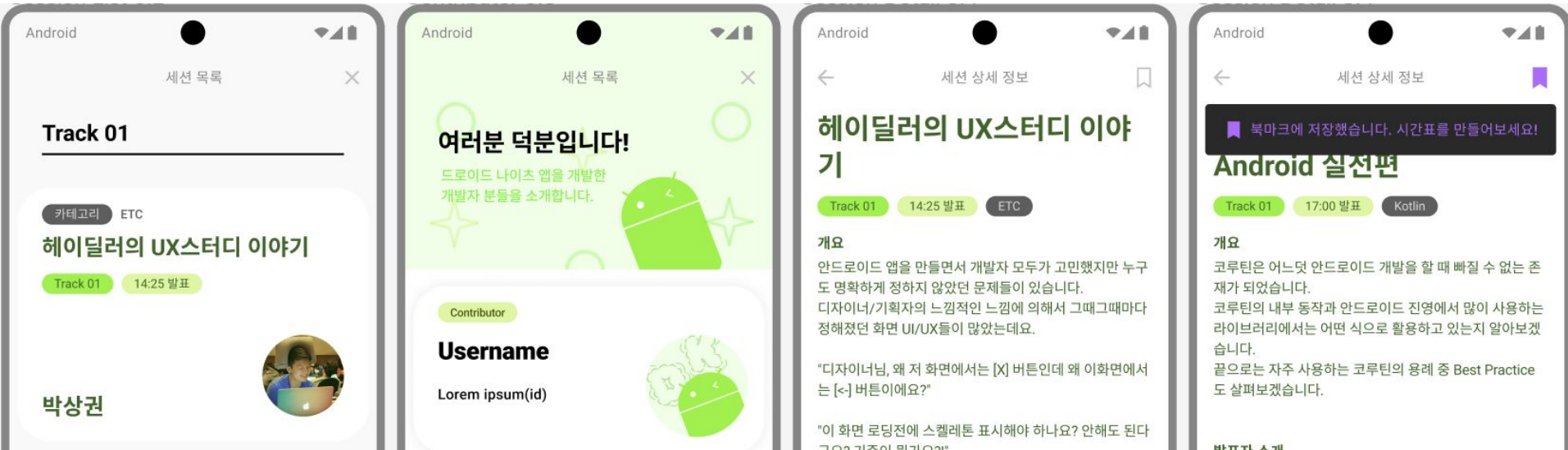
상대적으로 역사가 긴 선배 프레임워크 개발자들의
노하우를 참고할 수 있을 것이다.



사례로 알아보는 UI 컴포넌트 설계 팁

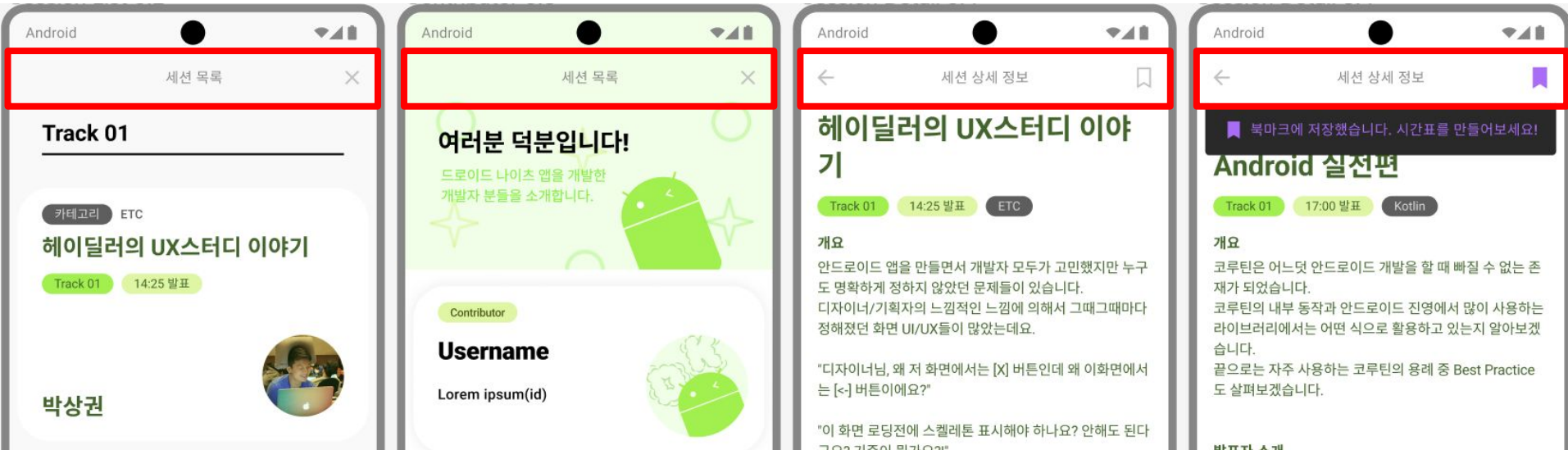
컴포넌트의 재사용성

비즈니스의 확장과 변경을 고려한 설계



컴포넌트의 재사용성

비즈니스의 확장과 변경을 고려한 설계



중복은 항상 제거되어야 할까?

❑ 외관상 중복된 코드라도 **수정의 이유가 다르면** 실제로는 중복이 아니다.

❑ 중복 제거를 강조하는 DRY(Don't repeat yourself) 원칙을 과도하게

적용함으로써 발생하는 의존성 문제와 조건문 추가로 인한 복잡성 증가는 오히려

유지보수를 어렵게 할 수 있다.

키노트

카테고리

키노트

키노트

10:40 발표

Track 01

카테고리 Android

Compose UI 컴포넌트 설계와 테스트

Track 01

13:55 발표

북마크

@Composable

fun KnightsTopAppBar(
 @StringRes titleRes: Int,
 ...
 isSessionDetail: Boolean,
) {
 ...
 Row(Modifier.align(Alignment.CenterEnd)) {
 if (isSessionDetail) BookmarkToggleButton(...)
 }
}

키노트

카테고리

키노트

키노트

10:40 발표

Track 01

카테고리 Android

Compose UI 컴포넌트 설계와 테스트

Track 01

13:55 발표

북마크

```
@Composable
fun KnightsTopAppBar(
    @StringRes titleRes: Int,
    ...
    isSessionDetail: Boolean,
) {
    ...
    Row(Modifier.align(Alignment.CenterEnd)) {
        if (isSessionDetail) BookmarkToggleButton(...)
    }
}
```

키노트

카테고리

키노트

키노트

10:40 발표

Track 01

카테고리

Android

Compose UI 컴포넌트 설계와 테스트

Track 01

13:55 발표

북마크

X 컴포넌트끼리 강하게 결합,
직관적이지 않은 API 인터페이스

@Composable

fun KnightsTopAppBar(
 @StringRes titleRes: Int,
 ...
 isSessionDetail: Boolean,
 isTimetable: Boolean,
 isSomethingElse: Boolean,
) {
 ...
 Row(Modifier.align(Alignment.CenterEnd)) {
 if (isSessionDetail) BookmarkToggleButton(...)
 if (isTimetable) EditModeButton(...)
 if (isSomethingElse) SomethingElseButton(...)
 }
}

중복은 항상 제거되어야 할까?

- ❑ 조건문 추가는 컴포넌트가 수정되어야 하는 이유가 다름을 의미한다.
 - ❑ 이는 중복 제거와 재사용 대상이 아니라 복잡성의 시작이다
- ❑ 조건문이 없는 컴포넌트를 만들기는 쉽지만 조건문의 추가가 컴포넌트에 어떤 영향을 주는지 이해해야 한다.
- ❑ 재사용성을 고려할 때는 공통적인 요소만을 추출하고, 컴포넌트 고유의 속성은 외부에서 전달하는 방식을 고려해야 한다.

키노트

카테고리

키노트

키노트

10:40 발표

Track 01

카테고리 Android

Compose UI 컴포넌트 설계와 테스트

Track 01

13:55 발표

북마크



Composition(조합)

```
@Composable
fun KnightsTopAppBar(
    @StringRes titleRes: Int,
    ...
    actionButtons: @Composable () -> Unit = {},
) {
    ...
    Row(Modifier.align(Alignment.CenterEnd)) {
        actionButtons()
    }
}
```


키노트

카테고리

키노트

키노트

10:40 발표

Track 01

카테고리 Android

Compose UI 컴포넌트 설계와 테스트

Track 01

13:55 발표

북마크

```
enum class TopAppBarNavigationType { Back, Close }
```

```
@Composable
```

```
fun KnightsTopAppBar(
```

```
    @StringRes titleRes: Int,
```

```
    ...
```

```
    navigationType: TopAppBarNavigationType,
```

```
    actionButtons: @Composable () -> Unit = {},
```

```
) {
```

```
    ...
```

```
    if (navigationType == TopAppBarNavigationType.Back) {
```

```
        icon(
```

```
            Modifier.align(Alignment.CenterStart),
```

```
            Icons.AutoMirrored.Filled.ArrowBack
```

```
        )
```

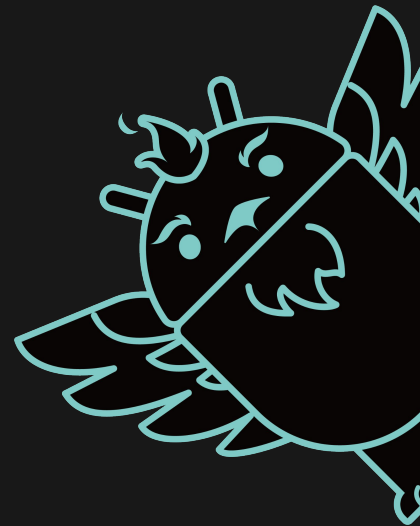
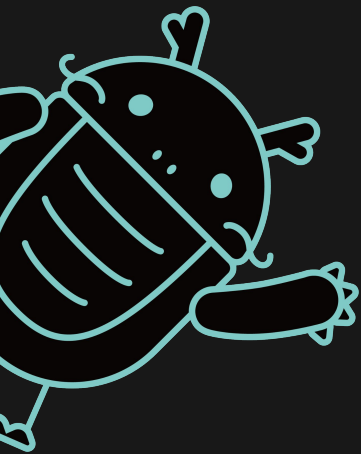
```
    }
```

```
    Row(Modifier.align(Alignment.CenterEnd)) {
```

```
        actionButtons()
```

```
    }
```

```
}
```



1. 재사용 가능한 컴포넌트
2. 결합도를 낮추고 응집도를 높인 설계
3. 직관적인 API 디자인

-> 변경사항에 유연하게 대처



사례로 알아보는 UI 컴포넌트 테스트

Stateful vs Stateless 컴포넌트

일반적으로 다음과 같이 구분한다.

- ❑ 자체적으로 상태를 생성/보유/수정하는 Stateful 컴포넌트
- ❑ 자체적으로 상태를 생성/보유/수정하지 않는 Stateless 컴포넌트


```
// Don't
```

```
@Composable
```

```
fun StatefulComponent() {  
    var username by remember { mutableStateOf("") }  
    TextField(  
        value = username,  
        onChange = { username = it },  
        label = { Text("Username") }  
    )  
}
```

상태를 공유하거나 재사용하기 어렵게 만들고,
컴포넌트 간의 결합도를 높인다.

// Don't

전제 조건(precondition) 설정이 어려워 테스트하기 까다롭다.

@Composable

```
fun StatefulComponent() {  
    var username by remember { mutableStateOf("") }  
    TextField(  
        value = username,  
        onValueChange = { username = it },  
        label = { Text("Username") }  
    )  
}
```

```
// Do
```

```
@Composable
```

```
fun StatefulComponent() {  
    var username by remember { mutableStateOf("") }  
    StatelessComponent(username, onChange = { username = it })  
}
```

```
@Composable
```

```
fun StatelessComponent(username: String, onChange: (String) -> Unit) {  
    TextField(  
        value = username,  
        onValueChange = onChange,  
        label = { Text("Username") }  
    )  
}
```

State Hoisting

상태를 상위 컴포넌트로 옮겨 여러 컴포넌트에서 공유하고,
UI의 상태를 쉽게 관리하고 테스트할 수 있다.

```
// Do
```

```
@Composable
```

```
fun StatefulComponent() {  
    var username by remember { mutableStateOf("") }  
    StatelessComponent(username, onChange = { username = it })  
}
```

```
@Composable
```

```
fun StatelessComponent(username: String, onChange: (String) -> Unit) {  
    TextField(  
        value = username,  
        onValueChange = onChange,  
        label = { Text("Username") }  
    )  
}
```

Stateful vs Stateless 컴포넌트

일반적으로 다음과 같이 구분한다.

- ❑ 자체적으로 상태를 생성/보유/수정하는 Stateful 컴포넌트
- ❑ 자체적으로 상태를 생성/보유/수정하지 않는 Stateless 컴포넌트

Stateful vs Stateless 컴포넌트

일반적으로 다음과 같이 구분한다.

- ❑ 자체적으로 상태를 생성/보유/수정하는 Stateful 컴포넌트
- ❑ 자체적으로 상태를 생성/보유/수정하지 않는 Stateless 컴포넌트

상태와 상태를 표시하는 UI를 분리하여 Stateless 컴포넌트를 위주로 테스트하자.

좀 더 일반적인 이야기

“그래서 테스트 가능한 컴포넌트가 뭔데?”

“컴포넌트 설계와 테스트가 어떤 연관이 있는걸까?”

컴포넌트 테스트 시나리오

컴포넌트 Contract 분석

컴포넌트에 기대하는 동작, 사용하기 적절한 상황

테스트가 필요한 항목 정의

- 컴포넌트가 입력받는 파라미터는 무엇인가?
- 어떤 전제 조건(precondition)을 설정할 수 있는가?
- 어떤 Side effect가 발생하는가?

불필요한 테스트 항목 제거

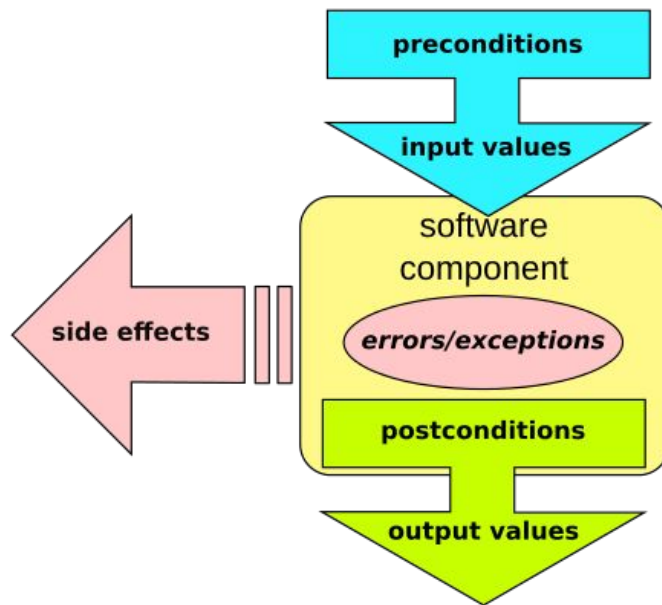
컴포넌트 테스트 시나리오

컴포넌트 Contract 분석

테스트가 필요한 항목 정의

불필요한 테스트 항목 제거

from: Design by Contract



컴포넌트 테스트 시나리오

컴포넌트 Contract 분석

컴포넌트 전체를 한 번에 테스트하기보다,
의미 있는 단위로 쪼개어진 컴포넌트를 각각 테스트한다.

테스트가 필요한 항목 정의

세션 상세 정보 화면 예시:

- `SessionDetailUiState`에 따라 로딩 UI 또는 세션 상세 UI(세션 제목, 내용, 발표자 정보, 태그, 시간)가 올바르게 표시되는지 확인
- 북마크 버튼 클릭 시 UI에 북마크 상태 변경이 반영되는지 확인
- 뒤로 가기 버튼 클릭 시 `onBackClick` 콜백 함수가 호출되는지 확인

...

불필요한 테스트 항목 제거

컴포넌트 테스트 시나리오

컴포넌트 Contract 분석

테스트가 필요한 항목 정의

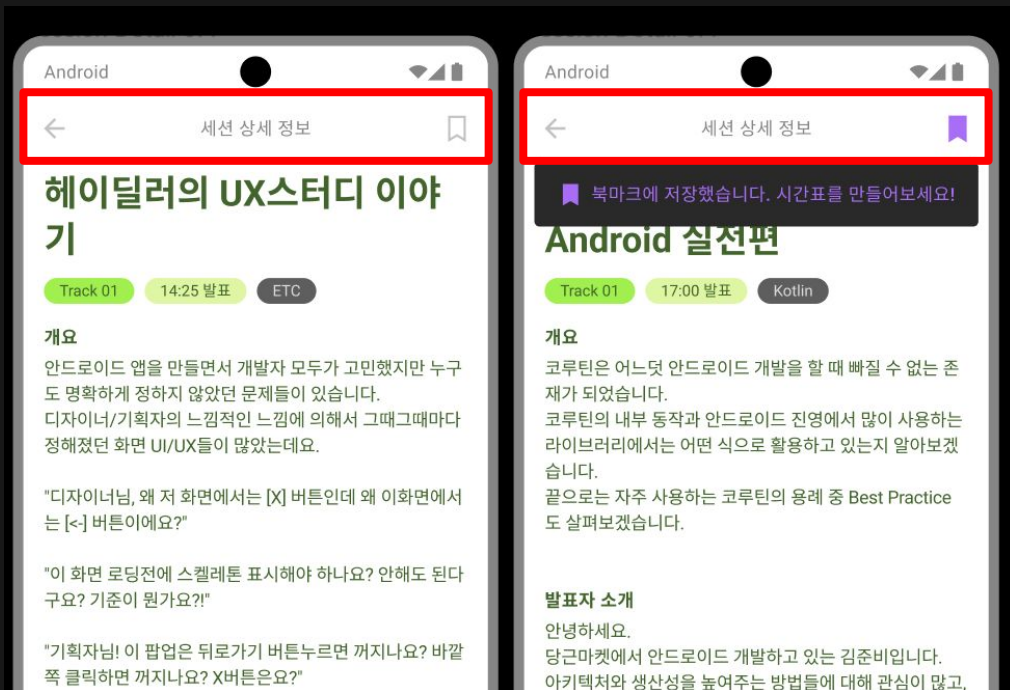
불필요한 테스트 항목 제거

테스트하지 않아도 되는 항목

- 불필요한 컴포넌트 내부 구현은 테스트하지 않는다.
(예: 이미지 로딩 방식)
- 일반적으로 Compose UI의 레이아웃 및 스타일에 적합한 테스트 방법이 따로 있다(Preview, Snapshot Testing 등)
(예: 각 버튼 컴포넌트가 선형으로 쌓였는지, 여백은 몇 dp인지)

Compose UI Testing 예시

시나리오: 북마크 아이콘 클릭 시 북마크 상태가 변경된다



1. 테스트 환경 세팅

```
@get:Rule  
val composeTestRule = createComposeRule()
```

```
@get:Rule
val composeTestRule = createComposeRule()
```

```
@Test
```

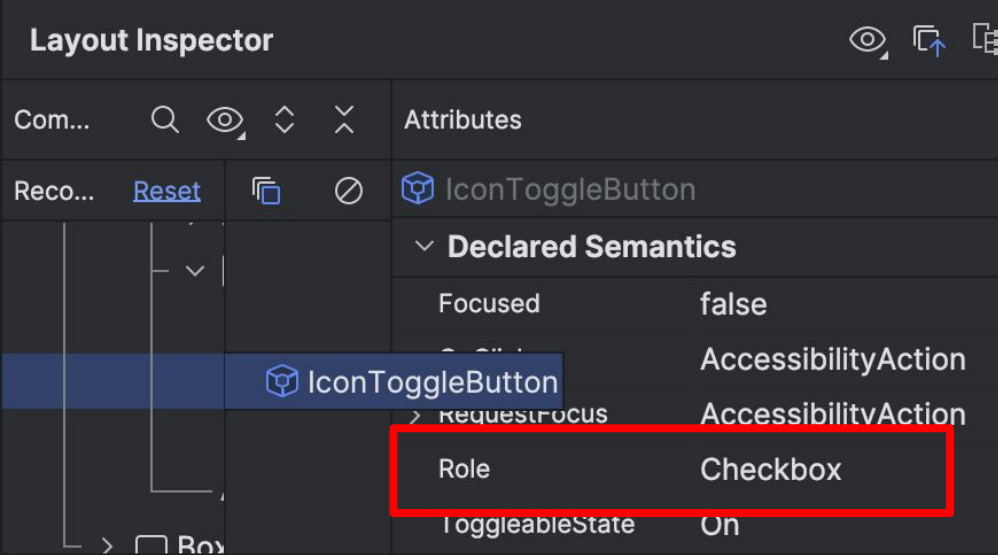
```
fun 북마크_아이콘_클릭_시_북마크_상태가_변경된다() {
    var bookmarked by mutableStateOf(false)
    composeTestRule.setContent {
        KnightsTheme {
            SessionDetailTopAppBar(
                bookmarked = bookmarked,
                onClickBookmark = { bookmarked = !bookmarked },
                onBackClick = { }
            )
        }
    }
}
```

1. 테스트 환경 세팅

2. 테스트를 위한 전제 조건 작성

1. 테스트 환경 세팅
2. 테스트를 위한 전제 조건 작성
3. 테스트 대상 레이아웃 찾기

```
@Test
fun 북마크_아이콘_클릭_시_북마크_상태가_변경된다() {
    var bookmarked by mutableStateOf(false)
    composeTestRule.setContent { ... }
    val bookmarkCheckbox = SemanticsMatcher.expectValue(
        SemanticsProperties.Role, Role.Checkbox
    )
    composeTestRule.onNode(bookmarkCheckbox)
        .performClick()
}
```



1. 테스트 환경 세팅
2. 테스트를 위한 전제 조건 작성
3. 테스트 대상 레이아웃 찾기

```
expectValue(  
    box
```

```
composeTestRule.onNode(bookmarkCheckbox)  
    .performClick()  
}
```

Layout Inspector -> Semantics Role

컴포넌트의 식별자를 명시하지 않더라도
컴포넌트의 역할 기반으로 구현체 가져오기

@Test

```
fun 북마크_아이콘_클릭_시_북마크_상태가_변경된다() {
```

```
    var bookmarked by mutableStateOf(false)
```

```
    composeTestRule.setContent { ... }
```

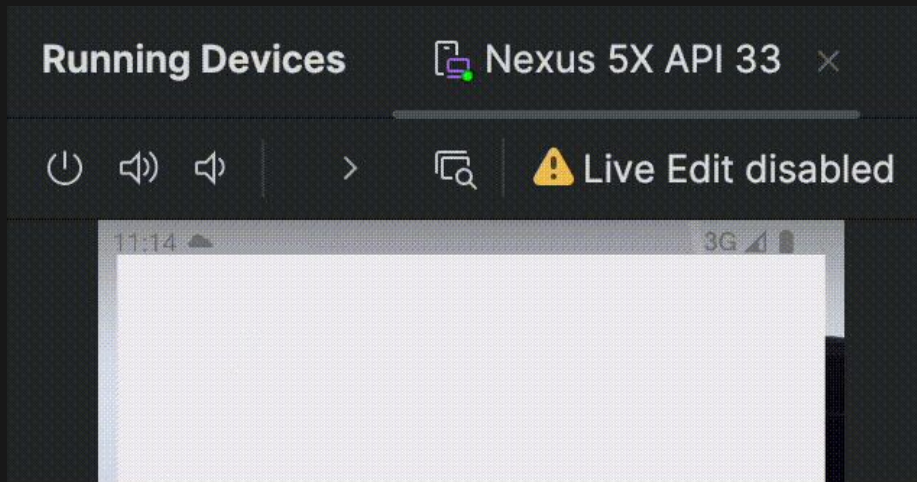
```
    val bookmarkCheckbox = SemanticsMatcher.expectValue(  
        SemanticsProperties.Role, Role.Checkbox  
    )
```

```
    composeTestRule.onNode(bookmarkCheckbox)  
        .performClick()  
        .assertIsOn()
```

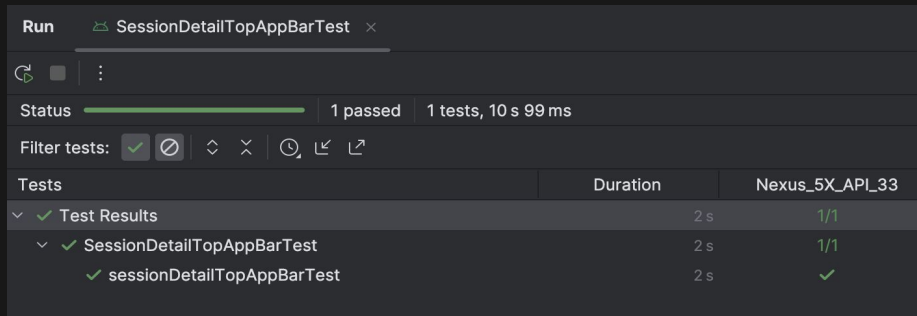
```
    assert(bookmarked)
```

```
}
```

1. 테스트 환경 세팅
2. 테스트를 위한 전제 조건 작성
3. 테스트 대상 레이아웃 찾기
4. 검증부 (Assertion)

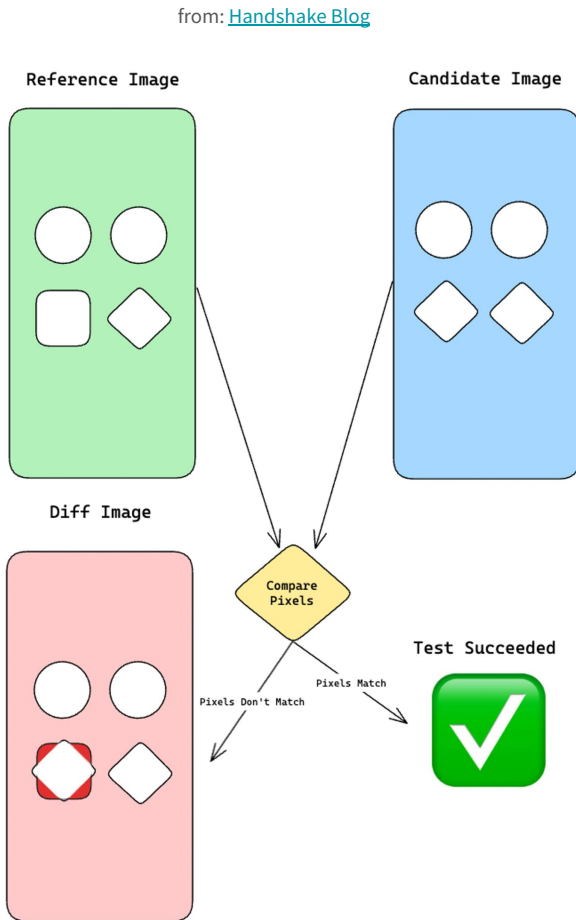


1. 테스트 환경 세팅
2. 테스트를 위한 전제 조건 작성
3. 테스트 대상 레이아웃 찾기
4. 검증부 (Assertion)
5. 실행 결과 확인



Screenshot Testing

- ❑ UI의 시각적 모습이 예상대로 표시되는지 검증
- ❑ 여러 해상도나 디바이스별 검증에 적합

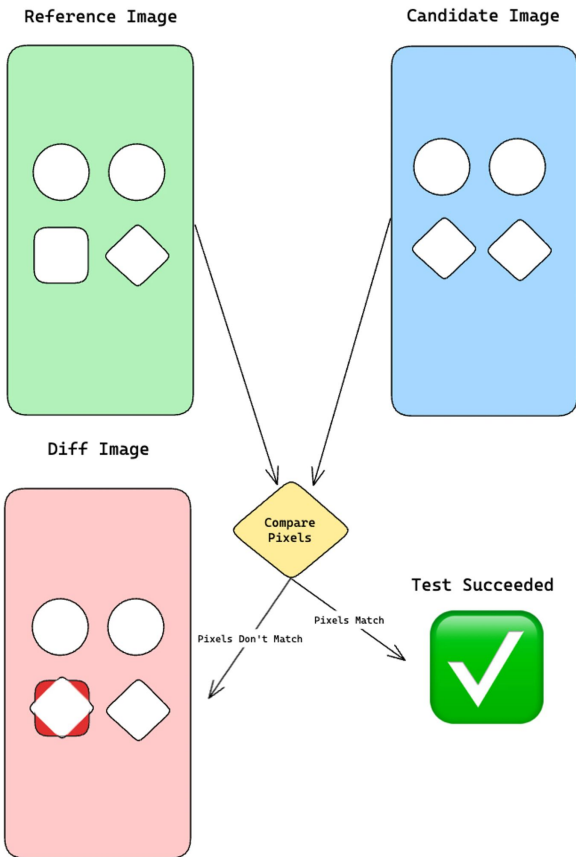


Screenshot Testing

적절한 단위로 쪼개어진 컴포넌트별 테스트는

- ❑ 한 번 작성된 Screenshot 테스트를
여러 테스트 케이스에서 활용 가능
- ❑ 업데이트 범위를 최소화
- ❑ 디자인 구현 정확도를 높임(디자인 시스템)

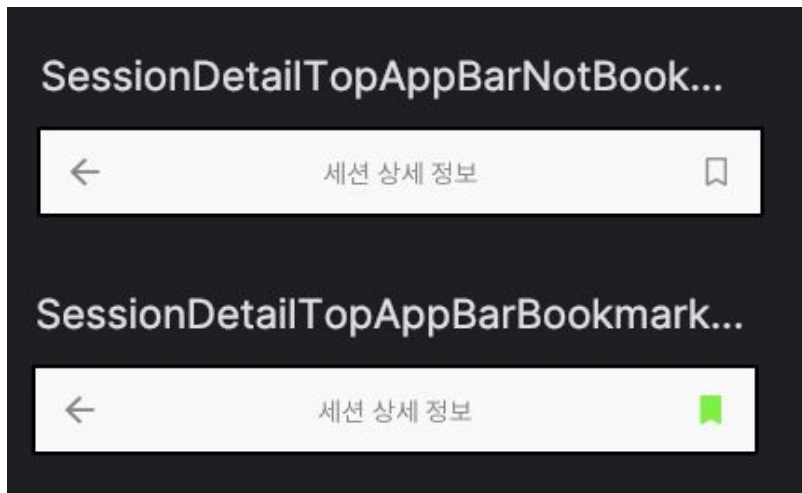
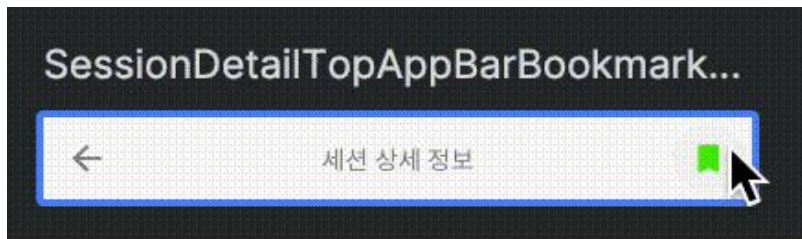
from: [Handshake Blog](#)



상황에 따라 Preview만으로도 충분

- ❑ 모든 시나리오에 UI 테스트 작성은 현실적으로 불가
- ❑ UI의 레이아웃 및 스타일 검증은 Preview만으로도 충분할 수 있음
- ❑ **Stateless** 컴포넌트로 구성하면 Precondition 설정 가능

↓ Interactive Mode



Compose Preview Screenshot Testing

from: [Google I/O 2024](#)

❑ Preview 기반 Reference 이미지 생성

❑ Screenshot Testing의 장점과

Preview의 장점을 결합

❑ 아직 실험적 단계

```
@Preview
@Composable
fun ForYouScreenPopulatedAndLoading() {
    NiaTheme {
        ForYouScreen(
            isSyncing = false,
            onboardingUiState = NotShown,
            feedState = Success( ... )
        )
    }
}
```

01 Add your Compose Previews to src/ScreenshotTest/

```
// Generates reference images for your tests
$ ./gradlew updateReleaseScreenshotTest

// Generates a screenshot test report
$ ./gradlew releaseScreenshotTest
```

02 Run the task to generate your reference images; run the task to generate a diff report after UI updates.

Actual image Diff image

```
@DevicePreviews
@FontScalePreviews
@ThemePreviews
@Composable
fun ForYouScreenPopulatedAndLoading() {
    ...
}
```

03 View the HTML test report to visually detect any changes to your app's UI.

(Recommended) Use multi-previews to scale your testing across screen sizes, themes, and more.

Compose Preview Screenshot Testing

Class `com.google.samples.apps.nowinandroid.feature.foryou.ForYouScreenshotTests`

[all](#) > [com.google.samples.apps.nowinandroid.feature.foryou](#) > ForYouScreenshotTests

8	1	0	0.562s
tests	failures	skipped	duration

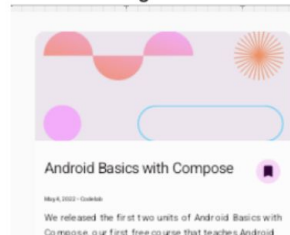
87%
successful

Failed tests

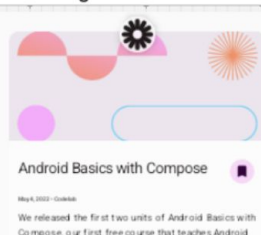
Tests

ForYouScreenPopulatedAndLoading

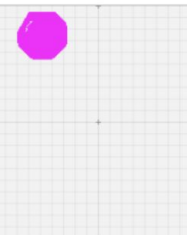
Reference Image



Actual Image



Diff Image



Compose UI 테스트 방법 및 도구 정리

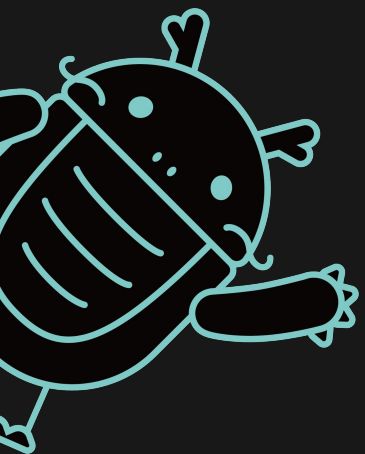
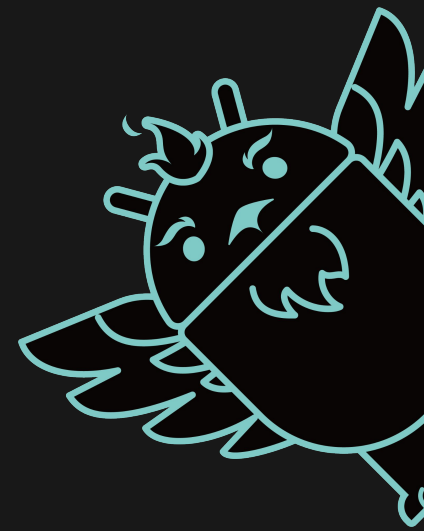
- ❑ Compose UI Testing
- ❑ Screenshot Testing
- ❑ Compose Preview

Compose UI 테스트 방법 및 도구 정리

- ❑ Compose UI Testing
- ❑ Screenshot Testing
- ❑ Compose Preview

테스트 용도와 현실적인 리소스에 따라 선택이 달라진다.

어떤 방법을 선택하더라도 적절한 단위의 컴포넌트 분리와
테스트 가능한 컴포넌트 설계는 중요한 요소다.

- 
- 
1. 재사용 가능한 컴포넌트
 2. 결합도를 낮추고 응집도를 높인 설계
 3. 직관적인 API 디자인
 4. 상태와 상태를 표시하는 UI 분리



-> 변경사항에 유연하게 대처
-> 테스트 가능한 컴포넌트

마무리

AOSP (Android Open Source Project)

<https://source.android.com>

AOSP는 안드로이드 운영 체제의 소스 코드를 관리하는 프로젝트로,
누구나 소스 코드를 열람하고 코드와 문서로 기여할 수 있다.



Search for code or files



Android

Android is a mobile operating system developed by Google

[VIEW PROJECT](#)



AndroidX

Libraries for Android development

[VIEW PROJECT](#)



Android Studio

Android Studio projects for building Android applications

[VIEW PROJECT](#)



Android-llvm

Android branch of the LLVM Project (a collection of modular and reusable compiler and toolchain technologies)

[VIEW PROJECT](#)

platform/frameworks/support > androidx-main > compose/material/material/src/androidInstrumentedTest/kotlin/androidx/compose/material/ ButtonTest.kt

Files

Outline

<|

ButtonTest.kt

Find

Links

View in

- material
- ▶ anchoredDraggable
- ▶ pullrefresh
- ▶ textfield
- AlertDialogScreenshotTest.kt
- AlertDialogTest.kt
- AppBarTest.kt
- AutoTestFrameClock.kt
- BackdropScaffoldTest.kt
- BadgeScreenshotTest.kt
- BadgeTest.kt
- BottomNavigationScreenshotTest.kt
- BottomNavigationTest.kt
- BottomSheetScaffoldTest.kt
- ButtonScreenshotTest.kt
- ButtonTest.kt**
- CardTest.kt
- CheckboxScreenshotTest.kt
- CheckboxTest.kt

```

130 }
137
138 @Test
139 fun canBeDisabled() {
140     val tag = "myButton"
141
142     rule.setMaterialContent {
143         var enabled by remember { mutableStateOf(true) }
144         val onClick = { enabled = false }
145         Box {
146             Button(modifier = Modifier.testTag(tag), onClick = onClick, enabled = enabled) {
147                 Text("Hello")
148             }
149         }
150     }
151     rule
152         .onNodeWithTag(tag)
153         // Confirm the button starts off enabled, with a click action
154         .assertHasClickAction()
155         .assertIsEnabled()
156         .performClick()
157         // Then confirm it's disabled with click action after clicking it
158         .assertHasClickAction()
159         .assertIsNotEnabled()
160 }
161
162 @Test
163 fun clickIsIndependentBetweenButtons() {
164     var button1Counter = 0
165     val button1OnClick: () -> Unit = { ++button1Counter }

```

AOSP - Compose 컴포넌트 테스트

이러한 테스트 코드는 단순히 기능을 검증하는 것 뿐만 아니라,
해당 컴포넌트가 어떻게 사용되어야 하는지,
어떤 결과물이 기대되는지에 대한 실질적인 문서 역할을 하기도 한다.

- ❑ [ButtonTest](#)
- ❑ [TextFieldTest](#)
- ❑ [LazyColumnTest](#)
- ❑ [SideEffectTests](#)

참고자료

- ❑ [안드로이드 공식 문서 - Compose](#)
- ❑ [React 공식 문서 - Thinking in react](#)
- ❑ [The Right Way to Test React Components - Stephen Scott](#)
- ❑ [변경에 유연한 컴포넌트 - jbee](#)

참고자료

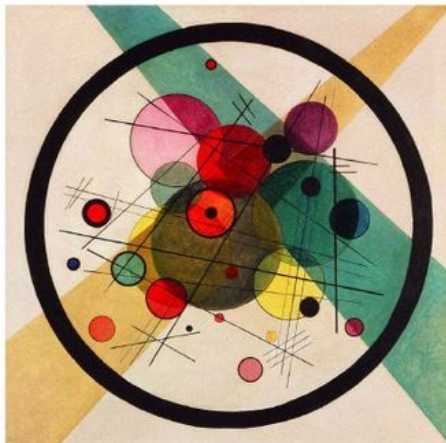
오브젝트

OBJECTS

코드로 이해하는
객체지향 설계

IT Leaders 2019

조영두 지음



하우북스

O'REILLY®

켄트 벡의 Tidy First?

더 나은 소프트웨어 설계를 위한
32가지 코드 정리법



시스템/하드웨어
설계 분야
아마존 베스트셀러

소프트웨어
설계의 선구자,
캐리 혼스타인
강력 추천

한빛미디어

켄트 벡 지음
한영희 옮김

참고자료

주제는 UI 컴포넌트 설계인데 참고자료 무슨 일?!? 😬

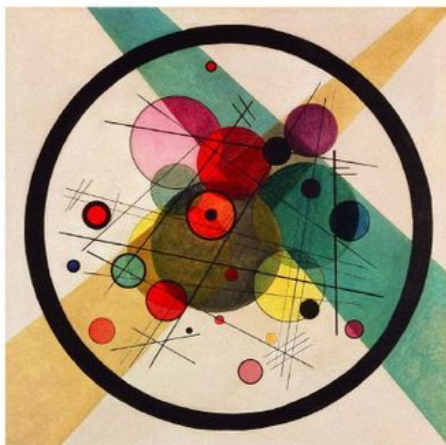
오브젝트

OBJECTS

코드로 이해하는
객체지향 설계

IT Leaders 2019

조영호 지음



하루복스

O'REILLY®

켄트 벅의 Tidy First?

더 나은 소프트웨어 설계를 위한
32가지 코드 정리법

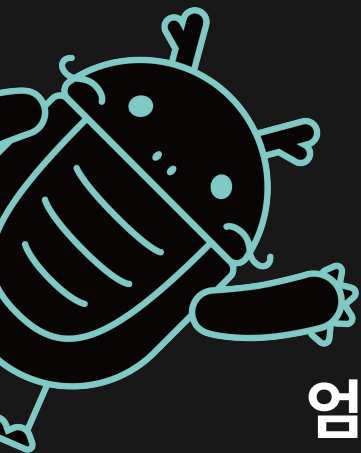


시스템/하드웨어
설계 분야
아마존 베스트셀러

소프트웨어
설계의 선구자,
캐리 혼스타인
강력 추천

한빛미디어

켄트 벅 지음
한영희 옮김



엄청난 아키텍처 패턴이나 방법론을 고려하기 전에,
변경사항에 대처하기 쉽고 유연한 설계부터 시작하자



즉, 기본기를 되새기자



Compose UI 컴포넌트 설계와 테스트

감사합니다

발표자료



Github

